

COMPUTER NETWORKS MODEL EXAM ANSWER

KEY

BATCH-1

- 1.) **Write a program with input "hello_client", the server listens for, and accepts, a single TCP connection; it reads all the data it can from that connection, and prints it to the screen; then it closes the connection.**

Ans:- AIM

To write a Java TCP server program that listens for a client connection, receives the message "hello_client", displays it, and closes the connection.

ALGORITHM

1. Create a ServerSocket on a specific port.
2. Wait for a client connection using accept().
3. Read data from the client using input stream.
4. Display the received message on the screen.
5. Close the connection and server socket.

Program

Server:-

```
import java.io.*;
import java.net.*;

public class TCPServer {
    public static void main(String[] args) {
        try {
            // Create server socket
            ServerSocket serverSocket = new ServerSocket(8080);
            System.out.println("Server is listening...");

            // Accept client connection
            Socket socket = serverSocket.accept();

            // Read data from client
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream())
            );

            String message = in.readLine();

            // Display message
            System.out.println("Message from client: " + message);

            // Close connections
            socket.close();
            serverSocket.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Client:-

```
import java.io.*;
import java.net.*;

public class TCPClient {
    public static void main(String[] args) {
        try {
            // Connect to server (localhost + port 8080)
            Socket socket = new Socket("localhost", 8080);

            // Send data to server
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

            String message = "hello_client";
            out.println(message);

            System.out.println("Message sent to server: " + message);

            // Close connection
            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

OUTPUT

Server:-

```
Server is listening...
Message from client: hello_client
```

Client:-

```
Message sent to server: hello_client
```

Result

The program was successfully executed and verified.

- 2.) Write the Echo client and server programs using UDP. The Echo clients should verify whether the text string they received from the server is the same text string that they sent or not.**

Ans:- AIM

To write Java UDP Echo client and server programs where the client sends a message and verifies whether the received message from the server is the same.

ALGORITHM

Server:

- 1. Create a DatagramSocket and bind to a port.**
- 2. Receive datagram packet from client.**
- 3. Extract message from packet.**
- 4. Send the same message back to client (echo).**
- 5. Close the socket.**

Client:

- 1. Create a DatagramSocket.**
- 2. Send a message to server using DatagramPacket.**
- 3. Receive echoed message from server.**
- 4. Compare sent and received messages.**
- 5. Display result and close socket.**

PROGRAM

SERVER:-

```
import java.net.*;

public class UDPEchoServer {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket(9876);
            byte[] receiveData = new byte[1024];
            byte[] sendData;

            System.out.println("UDP Server is running...");

            // Receive data
            DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
            socket.receive(receivePacket);

            String message = new String(receivePacket.getData(), 0, receivePacket.getLength());
            System.out.println("Received from client: " + message);

            // Echo back
            InetAddress clientAddress = receivePacket.getAddress();
            int clientPort = receivePacket.getPort();

            sendData = message.getBytes();
            DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, clientAddress, clientPort);
            socket.send(sendPacket);

            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

CLIENT:-

```
import java.net.*;

public class UDPEchoClient {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket();

            String message = "hello_client";
            byte[] sendData = message.getBytes();
            byte[] receiveData = new byte[1024];

            InetAddress serverAddress = InetAddress.getByName("localhost");

            // Send to server
            DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, serverAddress, 9876);
            socket.send(sendPacket);

            // Receive echo
            DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
            socket.receive(receivePacket);

            String receivedMessage = new String(receivePacket.getData(), 0, receivePacket.getLength());

            System.out.println("Sent: " + message);
            System.out.println("Received: " + receivedMessage);

            // Verification
            if (message.equals(receivedMessage)) {
                System.out.println("Verification: SUCCESS (Same message)");
            } else {
                System.out.println("Verification: FAILED (Different message)");
            }

            socket.close();
        }
    }
}
```

```
    } catch (Exception e) {  
        System.out.println(e);  
    }  
}
```

OUTPUT

SERVER:-

```
UDP Server is running...  
Received from client: hello_client
```

CLIENT:-

```
Sent: hello_client  
Received: hello_client  
Verification: SUCCESS (Same message)
```

- 3.) To implement a CHAT application using TCP
Socket communication for client and server. Both
can run in the same machine or different machines.

Ans:- AIM

To implement a chat application using TCP socket communication between client and server on the same or different machines.

ALGORITHM

Server:

- 1. Create ServerSocket and wait for client connection.**
- 2. Accept connection using accept().**
- 3. Create input and output streams.**
- 4. Receive message from client and send reply.**
- 5. Repeat until “exit” and close connection.**

Client:

- 1. Create Socket to connect to server.**
- 2. Create input and output streams.**
- 3. Send message to server.**
- 4. Receive reply from server.**
- 5. Repeat until “exit” and close connection.**

PROGRAM

SERVER:-

```

import java.io.*;
import java.net.*;

public class ChatServer {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(1234);
            System.out.println("Server waiting for connection...");

            Socket socket = serverSocket.accept();
            System.out.println("Client connected!");

            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader userInput = new BufferedReader(
                new InputStreamReader(System.in));

            String clientMsg, serverMsg;

            while (true) {
                // Receive from client
                clientMsg = in.readLine();
                System.out.println("Client: " + clientMsg);

                if (clientMsg.equalsIgnoreCase("exit")) break;

                // Send reply
                System.out.print("Server: ");
                serverMsg = userInput.readLine();
                out.println(serverMsg);

                if (serverMsg.equalsIgnoreCase("exit")) break;
            }
        }
    }
}

```

```

        socket.close();
        serverSocket.close();

    } catch (Exception e) {
        System.out.println(e);
    }
}
}

```

CLIENT:-


```

import java.io.*;
import java.net.*;

public class ChatClient {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 1234);
            System.out.println("Connected to server!");

            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader userInput = new BufferedReader(
                new InputStreamReader(System.in));

            String clientMsg, serverMsg;

            while (true) {
                // Send message
                System.out.print("Client: ");
                clientMsg = userInput.readLine();
                out.println(clientMsg);

                if (clientMsg.equalsIgnoreCase("exit")) break;

                // Receive reply
                serverMsg = in.readLine();
                System.out.println("Server: " + serverMsg);

                if (serverMsg.equalsIgnoreCase("exit")) break;
            }

            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

```

OUTPUT

Server:-

```

C:\Users\HP\Downloads\Computer Networks Practicals>java ChatServer
Server waiting for connection...
Client connected!
Client: hello
Server: hi
Client: how are you
Server: fine
Client: how are you
Server: fine
|

```

Client:-

```
C:\Users\HP\Downloads\Computer Networks Practicals>java ChatClient
Connected to server!
Client: hello
Server: hi
Client: how are you
Server: fine
Client: how are you
Server: fine
Client: |
```

- 4.) Write a program to implement DNS using UDP Sockets by getting any frame size based on the user request and send the frames to server.**

Ans:- AIM

To implement a DNS using UDP sockets where the client sends a domain name in frames based on user-defined frame size, and the server resolves and returns the IP address.

ALGORITHM

Server:

- 1. Create a DatagramSocket and wait for client packets.**
- 2. Receive frames and reconstruct the domain name.**
- 3. Resolve domain name to IP using InetAddress.**
- 4. Send the IP address back to client.**
- 5. Close the socket.**

Client:

1. Get domain name and frame size from user.
2. Split domain into frames based on frame size.
3. Send frames to server using UDP.
4. Receive IP address from server.
5. Display result and close socket.

PROGRAM

SERVER:-

```
1  import java.net.*;
2
3  public class DNSServer {
4      public static void main(String[] args) {
5          try {
6              DatagramSocket socket = new DatagramSocket(9876);
7              byte[] receiveData = new byte[1024];
8
9              System.out.println("DNS Server is running...");
10
11             StringBuilder domainBuilder = new StringBuilder();
12             InetAddress clientAddress = null;
13             int clientPort = 0;
14
15             // Receive all frames
16             while (true) {
17                 DatagramPacket packet = new DatagramPacket(receiveData, receiveData.length);
18                 socket.receive(packet);
19
20                 String part = new String(packet.getData(), 0, packet.getLength());
21
22                 if (part.equals("END")) {
23                     clientAddress = packet.getAddress();
24                     clientPort = packet.getPort();
25                     break;
26                 }
27
28                 domainBuilder.append(part);
29             }
30         }
31     }
```

```

30
31     String domain = domainBuilder.toString();
32     System.out.println("Received domain: " + domain);
33
34     // Resolve domain
35     InetAddress address = InetAddress.getByName(domain);
36     String ip = address.getHostAddress();
37
38     byte[] sendData = ip.getBytes();
39
40     DatagramPacket sendPacket = new DatagramPacket(
41         |     sendData, sendData.length, clientAddress, clientPort
42         |
43     );
44
45     socket.send(sendPacket);
46
47     socket.close();
48 } catch (Exception e) {
49     System.out.println(e);
50 }
51 }
52 }

```

CLIENT:

```

import java.net.*;
import java.util.Scanner;

public class DNSClient {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket();
            Scanner sc = new Scanner(System.in);

            System.out.print("Enter domain name: ");
            String domain = sc.nextLine();

            System.out.print("Enter frame size: ");
            int size = sc.nextInt();

            InetAddress serverAddress = InetAddress.getByName("localhost");

            // Send frames
            for (int i = 0; i < domain.length(); i += size) {
                String frame = domain.substring(i, Math.min(i + size, domain.length()));
                byte[] sendData = frame.getBytes();

                DatagramPacket packet = new DatagramPacket(
                    sendData, sendData.length, serverAddress, 9876
                );

                socket.send(packet);
            }

            // Send END signal
            byte[] endSignal = "END".getBytes();
            DatagramPacket endPacket = new DatagramPacket(
                endSignal, endSignal.length, serverAddress, 9876
            );
            socket.send(endPacket);

            // Receive IP
            byte[] receiveData = new byte[1024];
            DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
            socket.receive(receivePacket);

            String ip = new String(receivePacket.getData(), 0, receivePacket.getLength());

            System.out.println("Resolved IP: " + ip);

            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

```

OUTPUT:-

Server :-

```

C:\Users\HP\Downloads\Computer Networks Practicals>java DNSServer
DNS Server is running...
Received domain: google.com

```

Client:-

```
Enter domain name: google.com
Enter frame size: 3
Resolved IP: 142.250.205.206
```

5.) a) Write a program to implement to transfer the files using TCP socket.

b) Using Wireshark capture the FTP user name and password.

Ans:- a.) **AIM**

To implement file transfer between client and server using TCP sockets in Java.

ALGORITHM

Server:

1. Create ServerSocket and wait for connection.
2. Accept client connection.
3. Read file data from client using input stream.
4. Write data into a file.
5. Close connection.

Client:

1. Create Socket to connect to server.

2. Open file to be sent.
3. Read file data and send via output stream.
4. Ensure all data is transmitted.
5. Close connection.

PROGRAM

SERVER:-

```
import java.io.*;
import java.net.*;

public class FileServer {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(5000);
            System.out.println("Server waiting...");

            Socket socket = serverSocket.accept();
            System.out.println("Client connected");

            InputStream in = socket.getInputStream();
            FileOutputStream fos = new FileOutputStream("received.txt");

            byte[] buffer = new byte[1024];
            int count;

            while ((count = in.read(buffer)) > 0) {
                fos.write(buffer, 0, count);
            }

            System.out.println("File received successfully");

            fos.close();
            socket.close();
            serverSocket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

CLIENT:-

```
import java.io.*;
import java.net.*;

public class FileClient {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 5000);

            FileInputStream fis = new FileInputStream("send.txt");
            OutputStream out = socket.getOutputStream();

            byte[] buffer = new byte[1024];
            int count;

            while ((count = fis.read(buffer)) > 0) {
                out.write(buffer, 0, count);
            }

            System.out.println("File sent successfully");

            fis.close();
            socket.close();

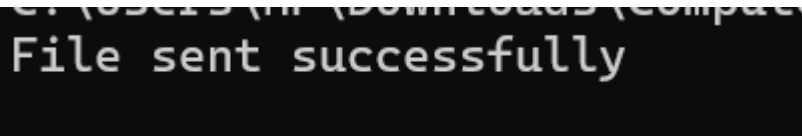
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

OUTPUT:-

Server:-

```
Server waiting...
Client connected
File received successfully
```

Client:-



File sent successfully

b.) AIM

To capture FTP username and password using a packet analyzer.

PROCEDURE

1. Install and open Wireshark on your system.
2. From the main screen, select the active network interface (such as Wi-Fi or Ethernet).
3. Click on **Start Capturing Packets** (shark fin icon).
4. In the filter bar, type:

ftp

and apply the filter to view only FTP-related packets.

5. Open any FTP client (Command Prompt or browser).
6. Connect to an FTP server using:

ftp ftp.dlptest.com

7. When prompted, enter:

- Username
- Password

8. While login is happening, Wireshark captures all packets.

9. Stop the capture after login is complete.
10. In Wireshark, look through the captured packets and identify packets containing:
 - USER
 - PASS
11. Click on a packet and inspect the **packet details pane**.
12. Under the **TCP stream / data section**, observe that:
 - Username appears as plain text
 - Password appears as plain text
13. You can also right-click a packet and select:
 **Follow → TCP Stream**
to clearly view the entire FTP communication.

Output:-

- The FTP protocol transmits data without encryption.
- Username and password are visible directly in captured packets:

USER testuser

PASS 12345

- 6.) Write the program for simulating the ARP protocol by reading an IP address and returning the MAC address.

Ans:- AIM

To simulate the ARP protocol by reading an IP address and returning the corresponding MAC address.

ALGORITHM

1. Initialize a table with predefined IP and MAC address pairs.
2. Read the IP address from the user.
3. Search for the IP in the table.
4. If found, display the corresponding MAC address.
5. Otherwise, display "IP address not found".

PROGRAM:-

```

import java.util.*;

public class ARPSimulation {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // ARP Table (IP → MAC)
        HashMap<String, String> arpTable = new HashMap<>();
        arpTable.put("192.168.1.1", "AA:BB:CC:DD:EE:01");
        arpTable.put("192.168.1.2", "AA:BB:CC:DD:EE:02");
        arpTable.put("192.168.1.3", "AA:BB:CC:DD:EE:03");
        arpTable.put("192.168.1.4", "AA:BB:CC:DD:EE:04");

        System.out.print("Enter IP address: ");
        String ip = sc.nextLine();

        // Lookup
        if (arpTable.containsKey(ip)) {
            System.out.println("MAC Address: " + arpTable.get(ip));
        } else {
            System.out.println("IP address not found in ARP table");
        }

        sc.close();
    }
}

```

OUTPUT:-

Case:1 :-

```

Enter IP address: 192.168.1.2
MAC Address: AA:BB:CC:DD:EE:02

```

Case:2 :-

```

Enter IP address: 192.168.1.10
IP address not found in ARP table

```

- 7.) Write a program for simulating RARP protocols using UDP.

Ans:- AIM

To simulate the RARP protocol using UDP sockets to obtain an IP address from a given MAC address.

ALGORITHM

Server:

1. Create a DatagramSocket and wait for client request.
2. Receive MAC address from client.
3. Search MAC in RARP table.
4. Send corresponding IP address to client.
5. Close socket.

Client:

1. Create a DatagramSocket.
2. Read MAC address from user.
3. Send MAC address to server.
4. Receive IP address from server.
5. Display result and close socket.

PROGRAM:-

SERVER:-

```
import java.net.*;
import java.util.HashMap;

public class RARPServer {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket(9876);
            byte[] receiveData = new byte[1024];
            byte[] sendData;

            // RARP Table (MAC → IP)
            HashMap<String, String> rarpTable = new HashMap<>();
            rarpTable.put("AA:BB:CC:DD:EE:01", "192.168.1.1");
            rarpTable.put("AA:BB:CC:DD:EE:02", "192.168.1.2");
            rarpTable.put("AA:BB:CC:DD:EE:03", "192.168.1.3");

            System.out.println("RARP Server is running...");

            // Receive MAC
            DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
            socket.receive(receivePacket);

            String mac = new String(receivePacket.getData(), 0, receivePacket.getLength());
            System.out.println("Received MAC: " + mac);

            // Lookup IP
            String ip = rarpTable.getOrDefault(mac, "IP not found");

            sendData = ip.getBytes();

            DatagramPacket sendPacket = new DatagramPacket(
                sendData, sendData.length,
                receivePacket.getAddress(),
                receivePacket.getPort()
            );

            socket.send(sendPacket);
            socket.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

CLIENT:-

```

import java.net.*;
import java.util.Scanner;

public class RARPCClient {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket();
            Scanner sc = new Scanner(System.in);

            System.out.print("Enter MAC Address: ");
            String mac = sc.nextLine();

            byte[] sendData = mac.getBytes();
            byte[] receiveData = new byte[1024];

            InetAddress serverAddress = InetAddress.getByName("localhost");

            // Send MAC to server
            DatagramPacket sendPacket = new DatagramPacket(
                sendData, sendData.length, serverAddress, 9876
            );
            socket.send(sendPacket);

            // Receive IP
            DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
            socket.receive(receivePacket);

            String ip = new String(receivePacket.getData(), 0, receivePacket.getLength());

            System.out.println("Assigned IP Address: " + ip);

            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

```

OUTPUT:-

SERVER:-

```

RARP Server is running...
Received MAC: AA:BB:CC:DD:EE:02

```

CLIENT:-

```

Enter MAC Address: AA:BB:CC:DD:EE:02
Assigned IP Address: 192.168.1.2

```

- 8.) Simulate Congestion Control Algorithms using NS and examine the performance of algorithm.

Ans:- **AIM**

To simulate congestion control algorithms and examine their performance using Java.

ALGORITHM

1. Initialize congestion window (cwnd) and threshold (ssthresh).
2. Increase cwnd using Slow Start (exponential growth).
3. When threshold is reached, switch to Congestion Avoidance (linear growth).
4. On packet loss, reduce cwnd and update ssthresh.
5. Repeat process and display changes.

PROGRAM:-


```

public class CongestionControl {
    public static void main(String[] args) {
        int cwnd = 1;           // Congestion Window
        int ssthresh = 16;      // Threshold
        int i;

        System.out.println("Round\tcwnd");

        for (i = 1; i <= 20; i++) {

            System.out.println(i + "\t" + cwnd);

            // Slow Start
            if (cwnd < ssthresh) {
                cwnd = cwnd * 2;
            }
            // Congestion Avoidance
            else {
                cwnd = cwnd + 1;
            }

            // Simulate packet loss at round 10
            if (i == 10) {
                System.out.println("Packet loss detected!");
                ssthresh = cwnd / 2;
                cwnd = 1; // reset
            }
        }
    }
}

```

OUTPUT:-

Round	cwnd
-------	------

1	1
---	---

2	2
---	---

3	4
---	---

4	8
---	---

5	16
---	----

6	17
---	----

7	18
---	----

8	19
---	----

9	20
---	----

10	21
----	----

Packet loss detected!

11	1
----	---

12	2
----	---

13	4
----	---

14	8
----	---

15	16
----	----

16	17
----	----

17	18
----	----

18	19
----	----

19	20
----	----

20	21
----	----

- 9.) Write a program to implement the Distance Vector Routing algorithm and find out the correct path from client to server where the packets send.

Ans:- AIM

To implement the Distance Vector Routing algorithm and determine the shortest path from client to server.

ALGORITHM

1. Initialize distance matrix for all nodes.
2. For each node, update distances using neighboring nodes.
3. Apply Bellman-Ford logic to update shortest paths.
4. Repeat until no changes occur.
5. Display shortest distance and path from source to destination.

PROGRAM

```

import java.util.Scanner;

public class DistanceVectorRouting {
    public static void main(String[] args) {
        int n, i, j, k;
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of nodes: ");
        n = sc.nextInt();

        int[][] cost = new int[n][n];
        int[][] dist = new int[n][n];

        System.out.println("Enter cost matrix:");
        for (i = 0; i < n; i++) {
            for (j = 0; j < n; j++) {
                cost[i][j] = sc.nextInt();
                dist[i][j] = cost[i][j];
            }
        }

        // Distance Vector Algorithm
        for (k = 0; k < n; k++) {
            for (i = 0; i < n; i++) {
                for (j = 0; j < n; j++) {
                    if (dist[i][j] > cost[i][k] + dist[k][j]) {
                        dist[i][j] = cost[i][k] + dist[k][j];
                    }
                }
            }
        }

        System.out.println("\nShortest distance matrix:");
        for (i = 0; i < n; i++) {
            for (j = 0; j < n; j++) {
                System.out.print(dist[i][j] + "\t");
            }
            System.out.println();
        }

        // Example: path from node 0 to last node
        System.out.println("\nShortest cost from node 0 to node " + (n-1) + " is: " + dist[0][n-1]);

        sc.close();
    }
}

```

OUTPUT:-

```

Enter number of nodes: 3
Enter cost matrix:
0 2 7
2 0 3
7 3 0

Shortest distance matrix:
0      2      5
2      0      3
5      3      0

Shortest cost from node 0 to node 2 is: 5

```

10.) Implementation of Link state routing algorithm and analyze the path where the packets are sent from client to server.

Ans:- AIM

To implement the Link State Routing algorithm and analyze the shortest path from client to server.

ALGORITHM

1. Initialize graph with cost between nodes.
2. Select source node and set initial distances.
3. Apply Dijkstra's algorithm to find shortest paths.
4. Update distances of neighboring nodes iteratively.
5. Display shortest path and cost from source to destination.

PROGRAM:-

```

import java.util.*;

public class LinkStateRouting {
    public static void main(String[] args) {
        int n, i, j;
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of nodes: ");
        n = sc.nextInt();

        int[][] graph = new int[n][n];

        System.out.println("Enter cost matrix:");
        for (i = 0; i < n; i++) {
            for (j = 0; j < n; j++) {
                graph[i][j] = sc.nextInt();
                if (graph[i][j] == 0 && i != j)
                    graph[i][j] = 999; // no connection
            }
        }

        System.out.print("Enter source node: ");
        int src = sc.nextInt();

        int[] dist = new int[n];
        boolean[] visited = new boolean[n];
        int[] parent = new int[n];

        // Initialize
        for (i = 0; i < n; i++) {
            dist[i] = graph[src][i];
            parent[i] = src;
            visited[i] = false;
        }

        dist[src] = 0;
        visited[src] = true;

        // Dijkstra Algorithm
        for (i = 0; i < n - 1; i++) {
            int min = 999, u = -1;

            for (j = 0; j < n; j++) {
                if (!visited[j] && dist[j] < min) {
                    min = dist[j];
                    u = j;
                }
            }

            visited[u] = true;
        }
    }
}

```

```

        for (j = 0; j < n; j++) {
            if (!visited[j] && (dist[u] + graph[u][j] < dist[j])) {
                dist[j] = dist[u] + graph[u][j];
                parent[j] = u;
            }
        }
    }

    // Print results
    System.out.println("\nNode\tDistance\tPath");

    for (i = 0; i < n; i++) {
        System.out.print(i + "\t" + dist[i] + "\t\t");
        printPath(parent, i, src);
        System.out.println();
    }

    sc.close();
}

// Function to print path
public static void printPath(int[] parent, int j, int src) {
    if (j == src) {
        System.out.print(src);
        return;
    }
    printPath(parent, parent[j], src);
    System.out.print(" -> " + j);
}
}

```

OUTPUT:-

```

Enter number of nodes: 4
Enter cost matrix:
0 2 0 1
2 0 3 2
0 3 0 4
1 2 4 0
Enter source node: 0

Node      Distance      Path
0          0          0
1          2          0 -> 1
2          5          0 -> 3 -> 2
3          1          0 -> 3

```

11.) To create Scenario and study the performance of CSMA/CD protocol through simulation. For the given data, use CRC. Verify the program for the cases:

- Without error.
- With error.

Ans:- AIM

To create a scenario and study the performance of CSMA/CD protocol using simulation and verify data transmission using CRC for both error and no-error cases.

ALGORITHM

1. Simulate channel sensing (idle/busy) and allow transmission.
2. If collision occurs, stop and retry (CSMA/CD concept).
3. Append CRC to data before transmission.
4. At receiver, recompute CRC and check remainder.
5. Display whether error is present or not.

PROGRAM:-


```

import java.util.Scanner;

public class CSMACD_CRC {

    // Function to perform XOR
    static String xor(String a, String b) {
        String result = "";
        for (int i = 1; i < b.length(); i++) {
            result += (a.charAt(i) == b.charAt(i)) ? '0' : '1';
        }
        return result;
    }

    // CRC Calculation
    static String crc(String data, String key) {
        int keyLen = key.length();
        String temp = data.substring(0, keyLen);
        int i = keyLen;

        while (i < data.length()) {
            if (temp.charAt(0) == '1')
                temp = xor(key, temp) + data.charAt(i);
            else
                temp = xor("0".repeat(keyLen), temp) + data.charAt(i);
            i++;
        }

        if (temp.charAt(0) == '1')
            temp = xor(key, temp);
        else
            temp = xor("0".repeat(keyLen), temp);

        return temp;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Data: ");
        String data = sc.next();

        System.out.print("Enter Generator: ");
        String key = sc.next();

        // Append zeros
        String appendedData = data + "0".repeat(key.length() - 1);

        // Generate CRC
        String remainder = crc(appendedData, key);
        String transmitted = data + remainder;
    }
}

```

```

        System.out.println("Transmitted Frame: " + transmitted);

        // CSMA/CD Simulation
        System.out.println("\nChannel Status: IDLE → Sending Data...");

        // Simulate error
        System.out.print("Introduce error? (yes/no): ");
        String choice = sc.next();

        String received = transmitted;

        if (choice.equalsIgnoreCase("yes")) {
            // Flip a bit to simulate error
            char[] arr = transmitted.toCharArray();
            arr[2] = (arr[2] == '0') ? '1' : '0';
            received = new String(arr);
            System.out.println("Error introduced in transmission!");
        }

        System.out.println("Received Frame: " + received);

        // Check CRC
        String check = crc(received, key);

        if (check.contains("1")) {
            System.out.println("Result: Error detected!");
            System.out.println("Collision detected → Retransmitting...");
        } else {
            System.out.println("Result: No error detected.");
        }

        sc.close();
    }
}

```

OUTPUT:-

Without Error :-

```

Enter Data: 1011
Enter Generator: 1101
Transmitted Frame: 1011100

Channel Status: IDLE ? Sending Data...
Introduce error? (yes/no): no
Received Frame: 1011100
Result: No error detected.

```

With Error:-

```
Enter Data: 1011
Enter Generator: 1101
Transmitted Frame: 1011100

Channel Status: IDLE ? Sending Data...
Introduce error? (yes/no): yes
Error introduced in transmission!
Received Frame: 1001100
Result: Error detected!
Collision detected ? Retransmitting...
```

12.) Implement Stop and Wait Protocol and Sliding Window Protocol in a C program and execute the same and display the result using Stop and Wait Protocol.

Ans:- **AIM**

To implement Stop and Wait Protocol and Sliding Window Protocol and display the result using Stop and Wait Protocol.

ALGORITHM

Stop & Wait:

1. Send one frame at a time.
2. Wait for acknowledgment (ACK).
3. If ACK received → send next frame.
4. If ACK lost → retransmit frame.

5. Repeat until all frames sent.

Sliding Window:

1. Send multiple frames within window size.
2. Receive ACKs for frames.
3. Slide window forward on ACK.
4. Retransmit lost frames if needed.
5. Continue until all frames sent.

PROGRAM:-

StopAndWait:-

```
import java.util.Random;

public class StopAndWait {
    public static void main(String[] args) {
        int frames = 5;
        Random rand = new Random();

        for (int i = 1; i <= frames; i++) {
            System.out.println("Sending Frame " + i);

            // Simulate ACK loss randomly
            boolean ackReceived = rand.nextBoolean();

            if (ackReceived) {
                System.out.println("ACK received for Frame " + i);
            } else {
                System.out.println("ACK lost for Frame " + i + " → Retransmitting...");
                i--; // resend same frame
            }
        }

        System.out.println("All frames sent successfully using Stop and Wait.");
    }
}
```

SlidingWindow:-

```

public class SlidingWindow {
    public static void main(String[] args) {
        int totalFrames = 8;
        int windowSize = 3;
        int sent = 0;

        while (sent < totalFrames) {
            System.out.println("\nSending frames:");

            for (int i = sent; i < sent + windowSize && i < totalFrames; i++) {
                System.out.println("Frame " + i + " sent");
            }

            System.out.println("Acknowledgment received for frames " + sent + " to " + (sent + windowSize - 1));

            sent += windowSize;
        }

        System.out.println("\nAll frames sent using Sliding Window Protocol.");
    }
}

```

OUTPUT:-

StopAndWait:-

```

Sending Frame 1
ACK lost for Frame 1 ? Retransmitting...
Sending Frame 1
ACK received for Frame 1
Sending Frame 2
ACK received for Frame 2
Sending Frame 3
ACK received for Frame 3
Sending Frame 4
ACK lost for Frame 4 ? Retransmitting...
Sending Frame 4
ACK received for Frame 4
Sending Frame 5
ACK lost for Frame 5 ? Retransmitting...
Sending Frame 5
ACK received for Frame 5
All frames sent successfully using Stop and Wait.

```

SlidingWindow:-

```
Sending frames:
Frame 0 sent
Frame 1 sent
Frame 2 sent
Acknowledgment received for frames 0 to 2

Sending frames:
Frame 3 sent
Frame 4 sent
Frame 5 sent
Acknowledgment received for frames 3 to 5

Sending frames:
Frame 6 sent
Frame 7 sent
Acknowledgment received for frames 6 to 8

All frames sent using Sliding Window Protocol.
```

13.) Write a HTTP web client program to download any webpage using TCP sockets.

Ans:- **AIM**

To write a Java HTTP web client program that downloads a webpage using TCP sockets.

ALGORITHM

1. Create a socket connection to the web server (port 80).
2. Send HTTP GET request for the webpage.

3. Read response from server.
4. Display or store the webpage content.
5. Close the connection.

PROGRAM:-

```
import java.io.*;
import java.net.*;

public class HTTPClient {
    public static void main(String[] args) {
        try {
            String host = "example.com";    // website
            int port = 80;

            // Create socket
            Socket socket = new Socket(host, port);

            // Send HTTP request
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            out.println("GET / HTTP/1.1");
            out.println("Host: " + host);
            out.println("Connection: close");
            out.println(); // blank line

            // Read response
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream())
            );

            String line;
            while ((line = in.readLine()) != null) {
                System.out.println(line);
            }

            socket.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

OUTPUT:-

```
HTTP/1.1 200 OK
Date: Wed, 06 May 2026 16:18:50 GMT
Content-Type: text/html
Transfer-Encoding: chunked
Connection: close
Server: cloudflare
Last-Modified: Fri, 01 May 2026 01:24:29 GMT
Allow: GET, HEAD
Accept-Ranges: bytes
Age: 10307
cf-cache-status: HIT
CF-RAY: 9f794d97ea5ca914-MAA

210
<!doctype html><html lang="en"><head><title>Example Domain</title><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></html>
```

- 14.) Write a program to implement the TCP echo client server for send the message input from the user and display the data received from the server.

Ans:- AIM

To implement a TCP Echo client-server program where the client sends a user-input message and receives the same message back from the server.

ALGORITHM

Server:

1. Create ServerSocket and wait for client connection.
2. Accept connection using accept().
3. Read message from client.
4. Send the same message back (echo).

5. Close connection.

Client:

1. Create Socket to connect to server.
2. Read message from user.
3. Send message to server.
4. Receive echoed message from server.
5. Display result and close connection.

PROGRAM:-

SERVER:-

```
import java.io.*;
import java.net.*;

public class TCPEchoServer {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(1234);
            System.out.println("Server waiting...");

            Socket socket = serverSocket.accept();
            System.out.println("Client connected");

            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

            String message = in.readLine();
            System.out.println("Received: " + message);

            // Echo back
            out.println(message);

            socket.close();
            serverSocket.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

CLIENT:-

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class TCPEchoClient {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 1234);

            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            Scanner sc = new Scanner(System.in);

            System.out.print("Enter message: ");
            String message = sc.nextLine();

            // Send to server
            out.println(message);

            // Receive echo
            String response = in.readLine();
            System.out.println("Echo from server: " + response);

            socket.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

OUTPUT:-

Server:-

```
Server waiting...
Client connected
Received: Hello
```

CLIENT:-

```
Enter message: Hello  
Echo from server: Hello
```

- 15.) Write a program for UDP echo client server for send the message input from the user and display the data received from the server.

Ans:- AIM

To implement a UDP Echo client-server program where the client sends a user-input message and receives the same message back from the server.

ALGORITHM

Server:

1. Create a DatagramSocket and bind to a port.
2. Receive datagram packet from client.
3. Extract message from packet.
4. Send the same message back (echo).
5. Close the socket.

Client:

1. Create a DatagramSocket.

2. Read message from user.
3. Send message to server using DatagramPacket.
4. Receive echoed message from server.
5. Display result and close socket.

PROGRAM:-

SERVER:-

```
import java.net.*;

public class UDPEchoServer1 {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket(9876);
            byte[] receiveData = new byte[1024];
            byte[] sendData;

            System.out.println("UDP Server waiting...");

            // Receive message
            DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
            socket.receive(receivePacket);

            String message = new String(receivePacket.getData(), 0, receivePacket.getLength());
            System.out.println("Received: " + message);

            // Echo back
            InetAddress clientAddress = receivePacket.getAddress();
            int clientPort = receivePacket.getPort();

            sendData = message.getBytes();
            DatagramPacket sendPacket = new DatagramPacket(
                sendData, sendData.length, clientAddress, clientPort
            );

            socket.send(sendPacket);

            socket.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

CLIENT:-

```

import java.net.*;
import java.util.Scanner;

public class UDPEchoClient1 {
    public static void main(String[] args) {
        try {
            DatagramSocket socket = new DatagramSocket();
            Scanner sc = new Scanner(System.in);

            System.out.print("Enter message: ");
            String message = sc.nextLine();

            byte[] sendData = message.getBytes();
            byte[] receiveData = new byte[1024];

            InetAddress serverAddress = InetAddress.getByName("localhost");

            // Send message
            DatagramPacket sendPacket = new DatagramPacket(
                sendData, sendData.length, serverAddress, 9876
            );
            socket.send(sendPacket);

            // Receive echo
            DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
            socket.receive(receivePacket);

            String response = new String(receivePacket.getData(), 0, receivePacket.getLength());

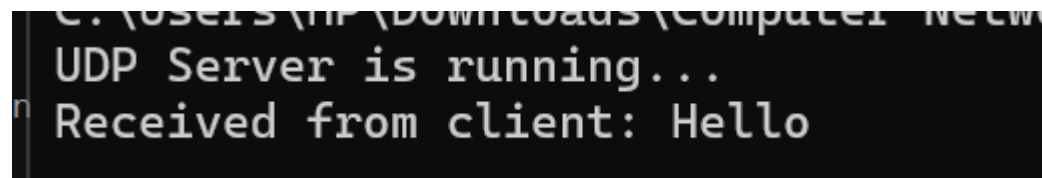
            System.out.println("Echo from server: " + response);

            socket.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

```

OUTPUT:-

SERVER:-

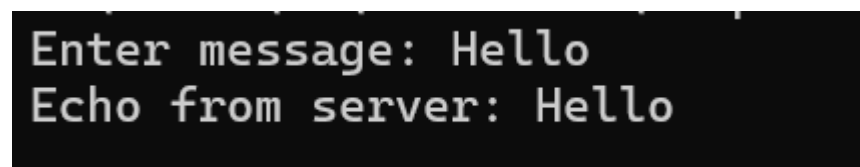


```

C:\Users\HP\Downloads\Computer Network
UDP Server is running...
Received from client: Hello

```

CLIENT:-



```

Enter message: Hello
Echo from server: Hello

```

16.) Analyze the TCP/UDP performance using Simulation tool and compare it with suitable data.

Ans:- AIM

To analyze the performance of TCP and UDP using simulation and compare them using suitable data.

ALGORITHM

1. Simulate data transmission using TCP (reliable).
2. Simulate data transmission using UDP (unreliable).
3. Measure parameters like delay, packet loss, and throughput.
4. Compare results for both protocols.
5. Display performance differences.

PROGRAM

```

import java.util.Random;

public class TCP_UDP_Simulation {
    public static void main(String[] args) {
        Random rand = new Random();

        int packets = 10;
        int tcpDelivered = 0;
        int udpDelivered = 0;

        System.out.println("Simulating TCP Transmission...");
        for (int i = 1; i <= packets; i++) {
            System.out.println("TCP Packet " + i + " sent");
            System.out.println("ACK received → Delivered");
            tcpDelivered++;
        }

        System.out.println("\nSimulating UDP Transmission...");
        for (int i = 1; i <= packets; i++) {
            System.out.println("UDP Packet " + i + " sent");

            // Simulate packet loss
            if (rand.nextInt(100) < 30) {
                System.out.println("Packet lost!");
            } else {
                System.out.println("Delivered");
                udpDelivered++;
            }
        }

        // Results
        System.out.println("\n--- Performance Analysis ---");
        System.out.println("TCP Delivered: " + tcpDelivered + "/" + packets);
        System.out.println("UDP Delivered: " + udpDelivered + "/" + packets);

        double tcpThroughput = (double) tcpDelivered / packets * 100;
        double udpThroughput = (double) udpDelivered / packets * 100;

        System.out.println("TCP Throughput: " + tcpThroughput + "%");
        System.out.println("UDP Throughput: " + udpThroughput + "%");
    }
}

```

OUTPUT:-

Simulating TCP Transmission...

TCP Packet 1 sent

ACK received ? Delivered

TCP Packet 2 sent

ACK received ? Delivered

TCP Packet 3 sent

ACK received ? Delivered

TCP Packet 4 sent

ACK received ? Delivered

TCP Packet 5 sent

ACK received ? Delivered

TCP Packet 6 sent

ACK received ? Delivered

TCP Packet 7 sent

ACK received ? Delivered

TCP Packet 8 sent

ACK received ? Delivered

TCP Packet 9 sent

ACK received ? Delivered

TCP Packet 10 sent

ACK received ? Delivered

Simulating UDP Transmission...

UDP Packet 1 sent

Packet lost!

UDP Packet 2 sent

Delivered

UDP Packet 3 sent

Delivered

UDP Packet 4 sent

Delivered

UDP Packet 5 sent

Packet lost!

UDP Packet 6 sent

Delivered

UDP Packet 7 sent

Packet lost!

UDP Packet 8 sent

Delivered

UDP Packet 9 sent

Packet lost!

UDP Packet 10 sent

Packet lost!

--- Performance Analysis ---

TCP Delivered: 10/10

UDP Delivered: 5/10

TCP Throughput: 100.0%

UDP Throughput: 50.0%

17.) Write a program to implement CRC error detection techniques. Identify the errors and display the error message.

Ans:- AIM

To implement CRC error detection technique, identify errors, and display an error message.

ALGORITHM

1. Read data and generator (divisor) from user.
2. Append zeros to data and compute CRC remainder.
3. Form transmitted frame (data + remainder).
4. At receiver, perform CRC division on received frame.
5. If remainder $\neq 0 \rightarrow$ error; else $\rightarrow$ no error.

PROGRAM

```

import java.util.Scanner;

public class CRC {

    // XOR operation
    static String xor(String a, String b) {
        String result = "";
        for (int i = 1; i < b.length(); i++) {
            result += (a.charAt(i) == b.charAt(i)) ? '0' : '1';
        }
        return result;
    }

    // CRC division
    static String mod2div(String data, String key) {
        int keyLen = key.length();
        String temp = data.substring(0, keyLen);

        for (int i = keyLen; i < data.length(); i++) {
            if (temp.charAt(0) == '1')
                temp = xor(key, temp) + data.charAt(i);
            else
                temp = xor("0".repeat(keyLen), temp) + data.charAt(i);
        }

        if (temp.charAt(0) == '1')
            temp = xor(key, temp);
        else
            temp = xor("0".repeat(keyLen), temp);

        return temp;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Data: ");
        String data = sc.next();

        System.out.print("Enter Generator: ");
        String key = sc.next();

        // Sender side
        String appendedData = data + "0".repeat(key.length() - 1);
        String remainder = mod2div(appendedData, key);
        String transmitted = data + remainder;

        System.out.println("Transmitted Frame: " + transmitted);
    }
}

```

```

// Receiver side
System.out.print("Enter Received Frame: ");
String received = sc.next();

String check = mod2div(received, key);

if (check.contains("1")) {
    System.out.println("Error detected in received data!");
} else {
    System.out.println("No error detected.");
}

sc.close();
}
}

```

OUTPUT:-

CASE :1 :-

```

Enter Data: 1011
Enter Generator: 1101
Transmitted Frame: 1011100
Enter Received Frame: 1011100
No error detected.

```

CASE 2:-

```

Enter Data: 1011
Enter Generator: 1101
Transmitted Frame: 1011100
Enter Received Frame: 1001100
Error detected in received data!

```

18.) Write a program to make the TCP server to receive a request, to process the request, and to send back the response. The server program needs to accept the request string, change all lowercase letters to uppercase letters, and return the result.

Ans:- AIM

To implement a TCP server that receives a request string from a client, converts all lowercase letters to uppercase, and sends back the processed response.

ALGORITHM

Server:

1. Create ServerSocket and wait for client connection.
2. Accept client request using accept().
3. Read the input string from client.
4. Convert string to uppercase.
5. Send the result back and close connection.

Client:

1. Create Socket to connect to server.
2. Read input string from user.
3. Send request to server.

4. Receive processed response.
5. Display result and close connection.

PROGRAM

SERVER:-

```
import java.io.*;
import java.net.*;

public class TCPServerUpper {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(5000);
            System.out.println("Server waiting...");

            Socket socket = serverSocket.accept();
            System.out.println("Client connected");

            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

            // Receive request
            String message = in.readLine();
            System.out.println("Received: " + message);

            // Process (convert to uppercase)
            String result = message.toUpperCase();

            // Send response
            out.println(result);

            socket.close();
            serverSocket.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

CLIENT:-

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class TCPClientUpper {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 5000);

            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            Scanner sc = new Scanner(System.in);

            System.out.print("Enter message: ");
            String message = sc.nextLine();

            // Send request
            out.println(message);

            // Receive response
            String response = in.readLine();
            System.out.println("Response from server: " + response);

            socket.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

```

OUTPUT:-

Server:-

```

Server waiting...
Client connected
Received: hello world

```

CLIENT:-

```

Enter message: hello world
Response from server: HELLO WORLD

```

19.) Write the performance evaluation of any two routing protocol using any simulation tool and compare it with your own data.

Ans:- AIM

To evaluate the performance of two routing protocols using simulation and compare them using suitable data.

ALGORITHM

1. Select two routing protocols (Distance Vector and Link State).
2. Simulate packet transmission between nodes.
3. Measure parameters like delay, packet delivery, and overhead.
4. Record observations for both protocols.
5. Compare performance using collected data.

PROGRAM


```

import java.util.Random;

public class RoutingPerformance {
    public static void main(String[] args) {
        Random rand = new Random();

        int packets = 10;
        int dvDelivered = 0;
        int lsDelivered = 0;

        System.out.println("Distance Vector Routing Simulation:");
        for (int i = 1; i <= packets; i++) {
            if (rand.nextInt(100) < 80) { // 80% success
                dvDelivered++;
            }
        }

        System.out.println("Link State Routing Simulation:");
        for (int i = 1; i <= packets; i++) {
            if (rand.nextInt(100) < 95) { // 95% success
                lsDelivered++;
            }
        }

        System.out.println("\n--- Results ---");
        System.out.println("DV Delivered: " + dvDelivered + "/" + packets);
        System.out.println("LS Delivered: " + lsDelivered + "/" + packets);

        double dvPerf = (double) dvDelivered / packets * 100;
        double lsPerf = (double) lsDelivered / packets * 100;

        System.out.println("DV Performance: " + dvPerf + "%");
        System.out.println("LS Performance: " + lsPerf + "%");
    }
}

```

OUTPUT:-

```

Distance Vector Routing Simulation:
Link State Routing Simulation:

--- Results ---
DV Delivered: 9/10
LS Delivered: 9/10
DV Performance: 90.0%
LS Performance: 90.0%

```